# Lab 3 — C++ Functions: Definition, Calling, Multi-file Projects & OpenCV

3. Laboratuvar — C++ Fonksiyonları, Çok Dosyalı Projeler ve OpenCV

第三次上机实验：C++语言函数定义与调用

Dì-sān cì shàngjī shíyàn: C++ yǔyán hánshù dìngyì yǔ diàoyòng

Course / Ders / 课程: 程序设计思维与方法B (Chéngxù shèjì sīwéi yǔ fāngfǎ B) — Programming Thinking & Methods B

| Part | Content / İçerik / 内容 |
|---|---|
| 0 | Objectives / Amaçlar / 实验目的 |
| 1 | Exp. 1 Function definition & call / 函数的定义与调用 (summary + crtexe.c) |
| 2 | Exp. 2 Calling functions across files / 跨文件调用函数 (.h/.cpp, extern, static, C1083, C2086) |
| 3 | Exp. 3 Open-source case: OpenCV / 开源代码应用案例 (licenses, search paths, VS setup, drawing) |
| 4 | Other labs: recursion primes, guessing game, camera effects / 其他实验 |
| 5 | Bonus: default args, inline, overloading / 默认参数、内联、重载 |
| 6 | Error cheat-sheet, vocabulary, self-check / 错误速查、词汇、自测 |

All code in this guide was compiled and run (g++; OpenCV examples checked with OpenCV 4.6 using only APIs that also exist in 2.4.10).

# 0. Objectives / Deneyin amaçları / 实验目的

| # | 中文 + Pinyin | English | Türkçe |
|---|---|---|---|
| 1 | 掌握函数的定义与调用方法<br>Zhǎngwò hánshù de dìngyì yǔ diàoyòng fāngfǎ | Master how to define and call functions | Fonksiyon tanımlama ve çağırmayı öğrenmek |
| 2 | 掌握函数的参数传递与返回值概念<br>Zhǎngwò hánshù de cānshù chuándì yǔ fǎnhuízhí gàiniàn | Understand parameter passing and return values | Parametre aktarımı ve dönüş değeri kavramlarını anlamak |
| 3 | 掌握变量的存储类型与作用域概念<br>Zhǎngwò biànliàng de cúnchǔ lèixíng yǔ zuòyòngyù gàiniàn | Understand storage class (static, extern…) and scope | Değişkenlerin saklama sınıfı ve kapsam kavramlarını anlamak |
| 4 | 验证带有默认值的函数定义和调用<br>Yànzhèng dàiyǒu mòrènzhí de hánshù dìngyì hé diàoyòng | Try functions with default argument values | Varsayılan parametreli fonksiyonları denemek |
| 5 | 了解内联函数与函数重载概念<br>Liǎojiě nèilián hánshù yǔ hánshù chóngzài gàiniàn | Know inline functions and function overloading | Satır içi (inline) fonksiyonlar ve aşırı yüklemeyi tanımak |
| 6 | 初步掌握递归函数设计方法<br>Chūbù zhǎngwò dìguī hánshù shèjì fāngfǎ | Learn basic recursive function design | Özyinelemeli (recursive) fonksiyon tasarımını öğrenmek |
| 7 | 了解OpenCV库的配置和调用方法<br>Liǎojiě OpenCV kù de pèizhì hé diàoyòng fāngfǎ | Learn to configure and use the OpenCV library | OpenCV kütüphanesini yapılandırmayı ve kullanmayı öğrenmek |

(Objectives 4 and 5 are greyed out on the slide — not checked in class, but covered briefly in Part 5.)

# 1. Experiment 1 — Function definition & call / 实验1 函数的定义与调用

**EN** Already explained in detail in the first guide (*Lab3.1_Functions_EN_TR_ZH.pdf*). The slide adds one detail: in Task 4 the function is **foo2**, and it asks for **two methods: a global variable or parameter passing** (pointer / reference).

**TR** İlk rehberde ayrıntılı anlatıldı. Slayt bir ayrıntı ekliyor: 4. görevde fonksiyon **foo2**'dir ve **iki yöntem** istenir: **global değişken** veya **parametre aktarımı** (işaretçi / referans).

中文 详见第一份讲义。幻灯片补充：要求4中的子程序是 foo2，要求用两种方法：全局变量或者参数传递。

*Xiángjiàn dì-yī fèn jiǎngyì. Huàndēngpiàn bǔchōng: yāoqiú sì zhōng de zǐ chéngxù shì foo2, yāoqiú yòng liǎng zhǒng fāngfǎ: quánjú biànliàng huòzhě cānshù chuándì.*

## Who calls main()? / main() fonksiyonunu kim çağırır? / main函数由谁调用？

**EN** When you keep pressing F10/F11 after `return 0;`, the debugger jumps into an "unknown" file **crtexe.c**. This is normal: **main() is called by the C runtime** start-up code (`mainCRTStartup`), which prepares the program and then calls main. Press F5 to finish.

**TR** `return 0;` sonrası F10/F11'e basmaya devam edince hata ayıklayıcı **crtexe.c** dosyasına atlar. Bu normaldir: **main()'i C çalışma zamanı (runtime) başlangıç kodu çağırır** (`mainCRTStartup`). Bitirmek için F5'e basın.

中文 调试过程跳出未知程序 crtexe.c：这是正常的，main 函数由C运行时库的启动代码调用。按 F5 继续运行即可。

*Tiáoshì guòchéng tiàochū wèizhī chéngxù crtexe.c: zhè shì zhèngcháng de, main hánshù yóu C yùnxíngshí kù de qǐdòng dàimǎ diàoyòng. Àn F5 jìxù yùnxíng jíkě.*

# 2. Experiment 2 — Calling functions across files / 实验2 跨文件调用函数

## 2.1 Header vs source files / Başlık ve kaynak dosyaları / 头文件与源文件

**EN** **.h header** = declarations (functions, variables, types, macros); can be shared by many .cpp files. **.cpp source** = the real implementations; one file per module. Same name for .h/.cpp is a convention, not a rule. A .cpp includes a header with `#include` — the compiler must be able to **find** it.

**TR** **.h başlık dosyası** = bildirimler (fonksiyon, değişken, tip, makro); birçok .cpp tarafından paylaşılır. **.cpp kaynak dosyası** = gerçek uygulama; her modül için bir dosya. Aynı ad bir gelenektir, zorunlu değildir. .cpp başlığı `#include` ile ekler; derleyici dosyayı **bulabilmelidir**.

中文 头文件（.h）存放函数、变量、类型和宏的声明，可以被多个源文件共享；源文件（.cpp）包含具体实现，通常一个文件实现一个功能模块。同名是常见约定，不是强制要求。源文件通过#include包含头文件，要确保编译器能找到。

*Tóuwénjiàn cúnfàng hánshù, biànliàng, lèixíng hé hóng de shēngmíng, kěyǐ bèi duō gè yuánwénjiàn gòngxiǎng; yuánwénjiàn bāohán jùtǐ shíxiàn, tōngcháng yí gè wénjiàn shíxiàn yí gè gōngnéng mókuài. Tóngmíng shì chángjiàn yuēdìng, bú shì qiángzhì yāoqiú.*

## 2.2 .obj and .lib / Nesne ve kütüphane dosyaları / 目标文件与库文件

**EN** Each .cpp is compiled into a binary **object file (.obj)** (unreadable in a text editor). Many .obj files can be packed into a **library (.lib)**. You can give a partner only the **.h + .obj/.lib** (not the .cpp): their project still builds, and your source code stays protected.

**TR** Her .cpp ikili bir **nesne dosyasına (.obj)** derlenir. Birçok .obj bir **kütüphane (.lib)** olarak paketlenebilir. Ortağınıza yalnızca **.h + .obj/.lib** verirseniz proje yine derlenir ve kaynak kodunuz korunur.

中文 源文件会被编译成目标文件（.obj），多个obj文件可以打包为库文件（.lib）。合作编程时只提供目标文件和头文件，也能完成编译，可以保护源文件的版权。

*Yuánwénjiàn huì bèi biānyì chéng mùbiāo wénjiàn (.obj), duō gè obj wénjiàn kěyǐ dǎbāo wéi kù wénjiàn (.lib). Hézuò biānchéng shí zhǐ tígōng mùbiāo wénjiàn hé tóuwénjiàn, yě néng wánchéng biānyì, kěyǐ bǎohù yuánwénjiàn de bǎnquán.*

## 2.3 The project & requirements / Proje ve görevler / 项目与实验要求

**EN** Functions **add5** and **add10** (in add5.h/.cpp, add10.h/.cpp at the project root) each update the global **i_globle** and the static **i_static**; a shared function **common** lives in the **common/** sub-folder. Use **extern** and **static** correctly.

**TR** **add5** ve **add10** fonksiyonları (proje kökündeki add5.h/.cpp, add10.h/.cpp) global **i_globle** ve statik **i_static** değişkenlerini günceller; ortak **common** fonksiyonu **common/** alt klasöründedir. **extern** ve **static** anahtar kelimelerini doğru kullanın.

中文 项目实现两个函数add5和add10，同时对全局变量i_globle和静态变量i_static进行操作；共用函数common存储在common子目录中。合理应用extern、static关键词达成项目意图。

*Xiàngmù shíxiàn liǎng gè hánshù add5 hé add10, tóngshí duì quánjú biànliàng i_globle hé jìngtài biànliàng i_static jìnxíng cāozuò; gòngyòng hánshù common cúnchǔ zài common zǐ mùlù zhōng. Hélǐ yìngyòng extern, static guānjiàncí dáchéng xiàngmù yìtú.*

| 中文 + Pinyin | English | Türkçe |
|---|---|---|
| 1. 解决错误 fatal error C1083：无法打开包括文件<br>Jiějué cuòwù C1083: wúfǎ dǎkāi bāokuò wénjiàn | Fix C1083: cannot open include file | C1083'ü çözün: include dosyası açılamıyor |
| 2. 解决错误 error C2086：重定义<br>Jiějué cuòwù C2086: chóng dìngyì | Fix C2086: redefinition | C2086'yı çözün: yeniden tanımlama |

| 中文 + Pinyin | English | Türkçe |
| --- | --- | --- |
| 3. 使用F10逐过程和F11逐语句调试，比较区别<br>Shǐyòng F10 zhú guòchéng hé F11 zhú yǔjù tiáoshì, bǐjiào qūbié | Debug with F10 / F11 and compare | F10 / F11 ile hata ayıklayın, karşılaştırın |
| 4. 调试过程中添加对变量的监视（watch），实时观察其变化<br>Tiáoshì guòchéng zhōng tiānjiā duì biànliàng de jiānshì, shíshí guānchá qí biànhuà | Add a Watch on variables and observe them live | Değişkenlere Watch (izleme) ekleyip canlı izleyin |
| 5. 程序结束时，i_globle和i_static都增加15<br>Chéngxù jiéshù shí, i_globle hé i_static dōu zēngjiā 15 | At the end both i_globle and i_static must have increased by 15 | Program sonunda i_globle ve i_static 15 artmış olmalı |
| 6. 观察没有源文件时的程序调试界面<br>Guānchá méiyǒu yuánwénjiàn shí de chéngxù tiáoshì jièmiàn | Observe the debugger when no source file exists | Kaynak dosya yokken hata ayıklayıcıyı gözlemleyin |

## 2.4 How to fix the two errors / İki hatanın çözümü / 两个错误的解决方法

**EN C1083** — the header is not where the compiler looks. Use the correct relative path `#include "common/common.h"`, **or** add the folder in Project → Properties → C/C++ → General → *`Additional Include Directories`*. Quotes `" "` search the source file's folder first; angle brackets `< >` search only system/configured folders.

**TR C1083** — başlık dosyası derleyicinin baktığı yerde değil. Doğru göreli yolu kullanın: `#include "common/common.h"` **veya** klasörü Proje → Özellikler → C/C++ → Genel → *`Ek Include Dizinleri`*'ne ekleyin. `" "` önce kaynak dosyanın klasörüne bakar; `< >` sadece sistem/ayarlı klasörlere bakar.

中文 C1083：编译器找不到头文件。使用正确的相对路径 #include "common/common.h"，或者在项目属性→C/C++→常规→附加包含目录中添加该目录。双引号先在源文件所在目录查找，尖括号只在系统库目录查找。

> *C1083: biānyìqì zhǎo bú dào tóuwénjiàn. Shǐyòng zhèngquè de xiāngduì lùjìng, huòzhě zài xiàngmù shǔxìng zhōng tiānjiā fùjiā bāohán mùlù. Shuāng yǐnhào xiān zài yuánwénjiàn suǒzài mùlù cházhǎo, jiān kuòhào zhǐ zài xìtǒng kù mùlù cházhǎo.*

**EN C2086 redefinition** — the same variable is *`defined`* twice, usually because `int i_globle = 0;` is written in a header that is included by several files (or included twice). Fix: in the header only **declare** it with `extern int i_globle;`, and **define** it in exactly one .cpp. Add `#pragma once` (or `#ifndef/#define/#endif`) to every header.

**TR C2086 yeniden tanımlama** — aynı değişken iki kez *`tanımlanmış`*; genelde `int i_globle = 0;` birden çok dosyaya eklenen bir başlıkta yazıldığı için. Çözüm: başlıkta sadece `extern int i_globle;` ile **bildirin**, tek bir .cpp'de **tanımlayın**. Her başlığa `#pragma once` ekleyin.

中文

C2086重定义：同一个变量被定义了两次，通常是因为在被多次包含的头文件中写了定义。解决：头文件中只用 extern 声明，在一个源文件中定义；每个头文件加上 #pragma once 防止重复包含。

> *C2086 chóng dìngyì: tóng yí gè biànliàng bèi dìngyì le liǎng cì. Jiějué: tóuwénjiàn zhōng zhǐ yòng extern shēngmíng, zài yí gè yuánwénjiàn zhōng dìngyì; měi gè tóuwénjiàn jiā shàng #pragma once fángzhǐ chóngfù bāohán.*

## 2.5 extern vs static / 外部与静态 (wàibù yǔ jìngtài)

| Keyword | English | Türkçe | 中文 + Pinyin |
|---|---|---|---|
| `extern int x;` | Declares a global defined in another file; no memory allocated. | Başka dosyada tanımlı globali bildirir; bellek ayırmaz. | 声明在其他文件中定义的全局变量，不分配内存。<br>shēngmíng zài qítā wénjiàn zhōng dìngyì de quánjú biànliàng |
| `static int x;` (file scope) | Global that is **private to its .cpp**; other files cannot see it. | Sadece **kendi .cpp dosyasına özel** global. | 只在本文件内可见的全局变量。<br>zhǐ zài běn wénjiàn nèi kějiàn |
| `static int x;` (in a function) | Local that **keeps its value** between calls (created once). | Çağrılar arasında **değerini korur**. | 静态局部变量，调用之间保持其值。<br>jìngtài júbù biànliàng, diàoyòng zhījiān bǎochí qí zhí |
| Trap / Tuzak / 陷阱 | `static int i_static;` in a **header** gives every .cpp its **own copy** → add5 and add10 change different copies, so you never see +15. | Başlıkta `static` → her .cpp'nin **kendi kopyası** olur; +15 görülmez. | 在头文件中定义static变量，每个源文件各有一份副本，结果不会是+15。<br>měi gè yuánwénjiàn gè yǒu yí fèn fùběn |

## 2.6 A working reference solution / Çalışan örnek çözüm / 参考答案

We do not have the course's Experiment 2 files, so this is **one correct design** that meets every requirement; your files may differ, but the ideas are the same. Output: `i_globle=15 i_static=15`.

common/common.h

```
#pragma once                // include guard: prevents C2086 redefinition
extern int i_globle;        // declaration only (no memory) - defined in common.cpp
void common(int n);         // adds n to i_globle and i_static
int  get_static();          // read i_static from outside its file
```

common/common.cpp

```
#include "common.h"
int i_globle = 0;           // the ONE definition of the global
static int i_static = 0;    // static: visible only inside common.cpp
void common(int n) { i_globle += n; i_static += n; }
int  get_static()  { return i_static; }
```

add5.h

```
#pragma once
void add5();
```

add5.cpp

```
#include "add5.h"
#include "common/common.h"   // relative path -> fixes C1083
void add5() { common(5); }
```

add10.h

```
#pragma once
void add10();
```

add10.cpp

```
#include "add10.h"
#include "common/common.h"
void add10() { common(10); }
```

main.cpp

```
#include <iostream>
#include "add5.h"
#include "add10.h"
#include "common/common.h"
int main() {
    add5();
    add10();
    std::cout << "i_globle=" << i_globle
              << " i_static=" << get_static() << std::endl;   // 15 15
    return 0;
}
```

## 2.7 Watch window & “no source” / İzleme penceresi / 监视窗口

**EN** While debugging, right-click a variable → **Add Watch** to see its value change live (or Debug → Windows → Watch). If you step (F11) into a function you only have as .obj/.lib, VS shows “**No source available**” / disassembly — you can still use F10 to step over it.

**TR** Hata ayıklarken değişkene sağ tıklayın → **Add Watch (İzleme Ekle)**. Sadece .obj/.lib olarak elinizde olan bir fonksiyona F11 ile girerseniz VS “**Kaynak yok**” veya makine kodu (disassembly) gösterir; F10 ile üzerinden geçebilirsiniz.

中文 调试时在变量处按右键，添加监视，观察变量值。如果没有源文件，F11进入函数时会显示“没有可用的源”或反汇编代码，可以用F10跳过。

> *Tiáoshì shí zài biànliàng chù àn yòujiàn, tiānjiā jiānshì, guānchá biànliàng zhí. Rúguǒ méiyǒu yuánwénjiàn, F11 jìnrù hánshù shí huì xiǎnshì “méiyǒu kěyòng de yuán” huò fǎnhuìbiān dàimǎ.*

# 3. Experiment 3 — Open-source case: OpenCV / 实验3 开源代码应用案例

**EN** **OpenCV** (Open Source Computer Vision Library) is a cross-platform computer-vision library started by **Intel in 1999**, under a BSD-type licence — free for commercial and non-commercial use. Used for image processing, feature/object detection and tracking, machine learning and deep learning.

**TR** **OpenCV**, **1999'da Intel** tarafından başlatılan, BSD tipi lisanslı, ticari ve ticari olmayan kullanım için ücretsiz, çok platformlu bir bilgisayarlı görü kütüphanesidir. Görüntü işleme, nesne algılama/takip, makine öğrenmesi ve derin öğrenmede kullanılır.

中文 OpenCV是一个跨平台计算机视觉库，最初由Intel公司在1999年发起建立，使用类BSD许可，对商业和非商业应用都免费。广泛用于图像处理、特征检测、目标检测和跟踪、机器学习和深度学习。

> *OpenCV shì yí gè kuà píngtái jìsuànjī shìjué kù, zuìchū yóu Intel gōngsī zài 1999 nián fāqǐ jiànlì, shǐyòng lèi BSD xǔkě, duì shāngyè hé fēi shāngyè yìngyòng dōu miǎnfèi.*

## 3.1 Open-source licences / Açık kaynak lisansları / 开源协议简介

**EN** History: early software was shared freely (C compiler, UNIX). After commercialisation, UNIX cost tens of thousands of dollars and lawsuits split it. **Richard Stallman** started the **GNU** project and the **GPL** (free software movement; he wrote **GCC**). Because "free" sounds like "no cost", it was renamed the **open-source** movement.

**TR** Tarihçe: ilk yazılımlar serbestçe paylaşılırdı (C derleyicisi, UNIX). Ticarileşince UNIX çok pahalı oldu ve davalar onu böldü. **Richard Stallman GNU** projesini ve **GPL**'i başlattı (**GCC**'yi yazdı). "Free" kelimesi "bedava" anlamına da geldiği için adı **açık kaynak** hareketi oldu.

中文 计算机产业初期代码免费共享；商业化后代码开始收费，UNIX版权诉讼造成版本分裂。Richard Stallman提出GNU计划和GPL协议，开启自由软件运动，贡献了GCC编译器。由于FREE容易误解为免费，后更名为开放软件运动。

> *Richard Stallman tíchū GNU jìhuà hé GPL xǔkě xiéyì, kāiqǐ le zìyóu ruǎnjiàn yùndòng. Yóuyú FREE róngyì wùjiě wéi miǎnfèi, hòu gēngmíng wéi kāifàng ruǎnjiàn yùndòng.*

| Licence | Must derived code stay open? | English | Türkçe | 中文 |
|---|---|---|---|---|
| GPL | Yes — whole program | Free to copy, distribute, charge, modify; derived works must also be GPL | Türev çalışmalar da GPL olmalı | 复制、传播、收费、修改自由；衍生作品也必须开源 |
| LGPL | Only changes to the library | Can link from closed-source code | Kapalı kaynak koddan bağlanabilir | 可被闭源程序调用，修改库本身需开源 |
| Mozilla (MPL) | Only modified files | File-level copyleft | Dosya düzeyinde copyleft | 只需开源修改过的文件 |
| BSD | No | Keep copyright notice; cannot use author's name to advertise | Telif notu korunmalı | 保留版权声明，不得用作者名义推广 |
| MIT | No | Keep copyright notice; almost no limits | Sadece telif notu | 几乎没有限制，只需保留版权声明 |
| Apache | No | Like BSD + patent grant; mark modified files | BSD gibi + patent izni | 类似BSD，并含专利授权，需说明修改 |

## 3.2 Where the compiler looks for files / Derleyici dosyaları nerede arar / 编译器搜索路径

**EN** 1) the source file's folder; 2) OS default folders (e.g. /usr/include on Linux); 3) user-defined paths (e.g. PATH environment variable); 4) paths set in the IDE/compiler options. OpenCV headers use <…>

includes, so its folders must be added to the search paths.

**TR** 1) kaynak dosyanın klasörü; 2) işletim sistemi varsayılanları (Linux'ta /usr/include); 3) kullanıcı yolları (PATH ortam değişkeni); 4) IDE/derleyici ayarlarındaki yollar. OpenCV başlıkları <…> kullandığı için klasörleri arama yollarına eklenmelidir.

中文 1.源文件所在目录；2.操作系统默认路径，如linux的include目录；3.用户自定义路径，如环境变量path；4.编译器和开发环境配置中指定的路径。OpenCV头文件大量使用尖括号，因此应将相关目录加入搜索路径。

*Yuánwénjiàn suǒzài mùlù; cāozuò xìtǒng mòrèn lùjìng; yònghù zìdìngyì lùjìng; biānyìqì hé kāifā huánjìng pèizhì zhōng zhǐdìng de lùjìng.*

## 3.3 Visual Studio 2010 setup for OpenCV 2.4.10 / VS ayarları / VS配置步骤

| Step | Where (Project → Properties) | Value |
|---|---|---|
| 0 | Run **opencv-2.4.10.exe**, extract to `C:\` | creates `C:\opencv\build\…` |
| 1 | VC++ Directories → **Executable Directories** / 可执行文件目录 | `C:\opencv\build\x86\vc10\bin` (.dll) |
| 2 | VC++ Directories → **Include Directories** / 包含目录 | `C:\opencv\build\include`<br>`C:\opencv\build\include\opencv`<br>`C:\opencv\build\include\opencv2` |
| 3 | VC++ Directories → **Library Directories** / 库目录 | `C:\opencv\build\x86\vc10\lib` |
| 4 | Linker → Input → **Additional Dependencies** / 附加依赖项 | paste everything from 附加依赖项列表**.txt** (e.g. `opencv_core2410d.lib; opencv_imgproc2410d.lib; opencv_highgui2410d.lib; …` — "d" = Debug) |
| 5 | "xxx.dll is missing" at run time / DLL bulunamadı / dll丢失 | Copy all .dll from `…\vc10\bin` into the **call_opencv** folder (next to main.cpp). Editing PATH needs a reboot, and lab PCs reset on reboot. |

**EN** x86 + vc10 = 32-bit build for Visual Studio 2010. Mixing x64 libraries or another vc version gives linker errors (LNK2019 / LNK1112).

**TR** x86 + vc10 = Visual Studio 2010 için 32-bit. x64 kütüphaneleri veya başka vc sürümünü karıştırmak bağlayıcı (linker) hatası verir.

中文 x86 + vc10 表示VS2010的32位版本，混用x64或其他版本的库会产生链接错误。

*x86 + vc10 biǎoshì VS2010 de 32 wèi bǎnběn, hùnyòng x64 huò qítā bǎnběn de kù huì chǎnshēng liànjiē cuòwù.*

## 3.4 Task: draw a rectangle, a line and text anywhere / 任意位置画矩形、画线、写文字

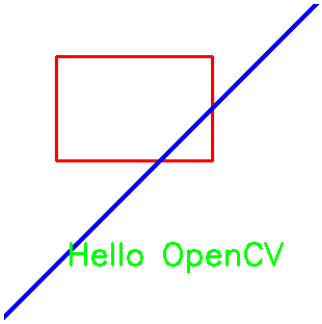
*Rènyì wèizhì huà jǔxíng, huà xiàn, xiě wénzì. Colours in OpenCV are* **(B, G, R)***, not RGB. Origin (0,0) is the* **top-left** *corner, y goes down.*

```
#include <opencv2/opencv.hpp>      // OpenCV 2.4.10: header in C:\opencv\build\include
using namespace cv;

int main() {
    Mat img = imread("lena.jpg");          // put lena.jpg next to main.cpp
    if (img.empty()) img = Mat(400, 400, CV_8UC3, Scalar(255, 255, 255));

    // rectangle: top-left, bottom-right, colour (B,G,R), thickness
    rectangle(img, Point(50, 50), Point(200, 150), Scalar(0, 0, 255), 2);
    // line: start, end
    line(img, Point(0, 300), Point(300, 0), Scalar(255, 0, 0), 3);
    // text: string, bottom-left corner, font, scale, colour, thickness
    putText(img, "Hello OpenCV", Point(60, 250),
            FONT_HERSHEY_SIMPLEX, 1.0, Scalar(0, 255, 0), 2);

    imshow("draw", img);
    waitKey(0);                            // wait for any key
    return 0;
}
```




Result on a blank canvas (when lena.jpg is missing). / Sonuç / 运行效果

## 3.5 Extension: a continuously moving shape / 扩展：实现一个连续运动的图形

*Kuòzhǎn: shíxiàn yí gè liánxù yùndòng de túxíng. Idea: in a loop — clear the frame, draw at the new position, show, wait 20 ms, update position.*

```
#include <opencv2/opencv.hpp>
using namespace cv;

int main() {
    int x = 30, dx = 4;                    // position and speed
    while (true) {
        Mat frame(300, 500, CV_8UC3, Scalar(255, 255, 255));  // clear
        circle(frame, Point(x, 150), 25, Scalar(0, 128, 255), -1); // -1 = filled
        imshow("moving", frame);
        x += dx;
        if (x > 475 || x < 25) dx = -dx;   // bounce at the edges
        if (waitKey(20) == 27) break;      // ~50 fps, ESC to quit
    }
    return 0;
}
```

# 4. Other experiments / Diğer deneyler / 其他实验

## 4.1 Textbook p.58, Lab 4.1 — Primes by recursion / Özyineleme ile asal sayılar / 递归法求素数

**EN** A recursive function needs a **base case** (stops) and a **recursive call** on a smaller problem. Here we try divisors d = 2, 3, … until d² > n.

**TR** Özyinelemeli fonksiyonun bir **temel durumu** (durma) ve daha küçük problemle **kendini çağırması** gerekir. Burada d = 2, 3, … bölenlerini d² > n olana dek deneriz.

中文 递归函数需要递归出口（终止条件）和递归调用。这里依次尝试除数 d = 2, 3, …，直到 d² 大于 n。

*Dìguī hánshù xūyào dìguī chūkǒu (zhōngzhǐ tiáojiàn) hé dìguī diàoyòng. Zhèlǐ yīcì chángshì chúshù d, zhídào d de píngfāng dàyú n.*

```
#include <iostream>
using namespace std;
// returns true if n has no divisor in [d, sqrt(n)]
bool isPrime(int n, int d = 2) {
    if (n < 2)      return false;   // 0,1 are not prime
    if (d * d > n)  return true;    // base case: no divisor found
    if (n % d == 0) return false;   // base case: divisor found
    return isPrime(n, d + 1);       // recursive call
}
int main() {
    for (int i = 1; i <= 50; i++)
        if (isPrime(i)) cout << i << " ";
    cout << endl;
    return 0;
}
```

Output / Çıktı / 输出: 2 3 5 7 11 13 17 19 23 29 31 37 41 43 47

## 4.2 Textbook p.59, Lab 4.4 — Guessing game / Sayı tahmin oyunu / 猜数游戏

```
#include <iostream>
#include <cstdlib>
#include <ctime>
using namespace std;
int main() {
    srand((unsigned)time(NULL));     // seed once
    int secret = rand() % 100 + 1;   // 1..100
    int guess, tries = 0;
    do {
        cout << "Guess (1-100): ";
        if (!(cin >> guess)) return 0;
        tries++;
        if (guess > secret)      cout << "Too big\n";
        else if (guess < secret) cout << "Too small\n";
    } while (guess != secret);
    cout << "Correct! tries = " << tries << endl;
    return 0;
}
```

## 4.3 Homework (not checked): camera effects / Ödev: kamera efektleri / 课后练习：摄像头特效

从摄像头读取画面连续显示，并实现反色、黑白、锐化、模糊等效果 — *Cóng shèxiàngtóu dúqǔ huàmiàn liánxù xiǎnshì, bìng shíxiàn fǎnsè, hēibái, ruìhuà, móhu děng xiàoguǒ. Press keys 0–4 to switch effect, ESC to quit.*

```
#include <opencv2/opencv.hpp>
using namespace cv;
```

```
int main() {
    VideoCapture cap(0);                    // camera 0
    if (!cap.isOpened()) return -1;
    Mat frame, out;
    int mode = 0;                           // keys: 0 normal 1 invert 2 gray 3 sharpen 4 blur
    while (cap.read(frame)) {
        switch (mode) {
        case 1: bitwise_not(frame, out); break;                  // 反色 invert
        case 2: cvtColor(frame, out, COLOR_BGR2GRAY); break;     // 黑白 gray
        case 3: {                                                 // 锐化 sharpen
            Mat k = (Mat_<float>(3,3) << 0,-1,0, -1,5,-1, 0,-1,0);
            filter2D(frame, out, -1, k); break; }
        case 4: GaussianBlur(frame, out, Size(15, 15), 0); break; // 模糊 blur
        default: out = frame;
        }
        imshow("camera", out);
        int key = waitKey(30);
        if (key == 27) break;
        if (key >= '0' && key <= '4') mode = key - '0';
    }
    return 0;
}
```

## 5. Bonus: default args, inline, overloading / 默认参数、内联函数、函数重载

**EN** **Default argument**: a parameter with a default value can be omitted (defaults go at the **right end**, written once — usually in the declaration). **inline**: asks the compiler to paste the body at the call site (for tiny functions). **Overloading**: same name, different parameter lists (return type alone is not enough).

**TR** **Varsayılan parametre**: varsayılan değeri olan parametre atlanabilir (sağ uçta olmalı). **inline**: derleyiciden gövdeyi çağrı yerine yapıştırmasını ister. **Aşırı yükleme**: aynı ad, farklı parametre listesi (sadece dönüş tipi yetmez).

中文 默认参数：带默认值的参数可以省略，必须从右向左设置。内联函数：建议编译器在调用处展开函数体。函数重载：函数名相同，参数列表不同（仅返回类型不同不能重载）。

*Mòrèn cānshù: dài mòrènzhí de cānshù kěyǐ shěnglüè, bìxū cóng yòu xiàng zuǒ shèzhì. Nèilián hánshù: jiànyì biānyìqì zài diàoyòng chù zhǎnkāi hánshùtǐ. Hánshù chóngzài: hánshù míng xiāngtóng, cānshù lièbiǎo bùtóng.*

```
#include <iostream>
using namespace std;
// default argument / varsayılan parametre / 默认参数
int area(int w, int h = 10) { return w * h; }
// inline function / satır içi fonksiyon / 内联函数
inline int square(int x) { return x * x; }
// overloading / aşırı yükleme / 函数重载
int    add(int a, int b)       { return a + b; }
double add(double a, double b) { return a + b; }
int main() {
    cout << area(5) << " " << area(5, 2) << endl;   // 50 10
    cout << square(7) << endl;                       // 49
    cout << add(1, 2) << " " << add(1.5, 2.5) << endl; // 3 4
    return 0;
}
```

## 6. Error cheat-sheet / Hata özeti / 常见错误速查

| Error | Meaning | Fix / Çözüm / 解决 |
|---|---|---|
| C3861 identifier not found | function used before declared / 找不到标识符 | add prototype or move definition up / 加声明 |
| C4244 conversion, possible loss of data | double → int / 可能丢失数据 | same types or explicit cast / 类型一致或强制转换 |
| C1083 cannot open include file | header not found / 无法打开包括文件 | correct path or Additional Include Dirs / 修改路径 |
| C2086 redefinition | defined twice / 重定义 | extern in .h, define in one .cpp, #pragma once |
| LNK2019 unresolved external symbol | declared but definition/lib missing / 无法解析的外部符号 | add .cpp to project or .lib to Additional Dependencies |
| LNK2005 already defined | global defined in a header used by 2 .cpp / 已经定义 | same fix as C2086 |
| “xxx.dll was not found” | runtime cannot find DLL / dll丢失 | copy DLLs next to the exe / 复制dll |

## 7. Vocabulary / Sözlük / 词汇表

| 中文 | Pinyin | English | Türkçe |
|---|---|---|---|
| 头文件 | tóuwénjiàn | header file | başlık dosyası |
| 源文件 | yuánwénjiàn | source file | kaynak dosyası |
| 目标文件 | mùbiāo wénjiàn | object file (.obj) | nesne dosyası |
| 库文件 | kù wénjiàn | library file (.lib) | kütüphane dosyası |
| 动态链接库 | dòngtài liànjiē kù | dynamic link library (.dll) | dinamik bağlantı kütüphanesi |
| 静态库 | jìngtài kù | static library | statik kütüphane |
| 跨文件 | kuà wénjiàn | across files | dosyalar arası |
| 模块化 | mókuàihuà | modularisation | modülerlik |
| 重用性 | chóngyòngxìng | reusability | yeniden kullanılabilirlik |
| 包含 | bāohán | include | dahil etmek |
| 重定义 | chóng dìngyì | redefinition | yeniden tanımlama |
| 存储类型 | cúnchǔ lèixíng | storage class | saklama sınıfı |
| 静态变量 | jìngtài biànliàng | static variable | statik değişken |
| 外部变量 | wàibù biànliàng | external (extern) variable | harici değişken |
| 监视 | jiānshì | watch | izleme |
| 参数传递 | cānshù chuándì | parameter passing | parametre aktarımı |
| 默认值 | mòrènzhí | default value | varsayılan değer |
| 内联函数 | nèilián hánshù | inline function | satır içi fonksiyon |
| 函数重载 | hánshù chóngzài | function overloading | fonksiyon aşırı yükleme |
| 递归 | dìguī | recursion | özyineleme |
| 素数 | sùshù | prime number | asal sayı |
| 猜数游戏 | cāi shù yóuxì | guessing game | sayı tahmin oyunu |
| 开源 | kāiyuán | open source | açık kaynak |
| 许可协议 | xǔkě xiéyì | licence | lisans |
| 计算机视觉 | jìsuànjī shìjué | computer vision | bilgisayarlı görü |

| 中文 | Pinyin | English | Türkçe |
|---|---|---|---|
| 图像处理 | túxiàng chǔlǐ | image processing | görüntü işleme |
| 配置 | pèizhì | configuration | yapılandırma |
| 包含目录 | bāohán mùlù | include directories | include dizinleri |
| 库目录 | kù mùlù | library directories | kütüphane dizinleri |
| 附加依赖项 | fùjiā yīlài xiàng | additional dependencies | ek bağımlılıklar |
| 链接器 | liànjiēqì | linker | bağlayıcı |
| 环境变量 | huánjìng biànliàng | environment variable | ortam değişkeni |
| 绝对路径 | juéduì lùjìng | absolute path | mutlak yol |
| 相对路径 | xiāngduì lùjìng | relative path | göreli yol |
| 矩形 | jǔxíng | rectangle | dikdörtgen |
| 摄像头 | shèxiàngtóu | camera | kamera |
| 反色 | fǎnsè | invert colours | renk ters çevirme |
| 锐化 | ruìhuà | sharpen | keskinleştirme |
| 模糊 | móhu | blur | bulanıklaştırma |

## 8. Self-check / Kendini test et / 自测题

| English | Türkçe | 中文 | Answer |
|---|---|---|---|
| What goes in a .h, what in a .cpp? | .h'ye ne, .cpp'ye ne yazılır? | 头文件和源文件各放什么？ | .h declarations, .cpp definitions / 声明 vs 实现 |
| How do you share a global across files? | Global değişken dosyalar arasında nasıl paylaşılır? | 如何跨文件共享全局变量？ | define in one .cpp + `extern` in .h |
| What does static do to a global? | static bir globali nasıl etkiler? | static<br>对全局变量有什么作用？ | file-private /<br>只在本文件可见 |
| Fix for "dll not found" in the lab? | Laboratuvarda "dll bulunamadı" çözümü? | 实验室中dll丢失怎么办？ | copy DLLs next to the exe / 复制dll |
| Colour order in OpenCV Scalar? | OpenCV Scalar renk sırası? | OpenCV颜色顺序？ | B, G, R |
| Two parts of every recursive function? | Her özyinelemeli fonksiyonun iki parçası? | 递归函数的两个要素？ | base case + recursive call / 出口+递归调用 |